

✓ Final Project: Substitution Cipher Decoding

Name: Julia Lee

Date: 10/11/2026

Course: CSCI 2011-51 Programming in Python

Objective

Decode an English-language message that has been encrypted using substitution cipher, using character frequency analysis, pattern recognition, and iterative mapping.

Approach

1. Count character frequencies in the encoded message
2. Make initial guesses based on common English words (the, and, this)
3. Apply the mapping and examine partially decoded text
4. Iteratively refine the mapping using context clues
5. Produce the final decoded message

```
1 from collections import Counter
2
3 encoded_msg = "jyn fg jggtwj djtfcn stf sjyn edcyjnc ia zy stes fjqtie z wzd
  owcfff gstf gsq sjyn edcyjnc gsfg mtgs tg gszi xjqcfcg owzm gstyf cycxtcf gz gtyq
  otgf ty gsq xcdhq jyn gsc wzd ntn edty jyn aczawc ntn lcjfg iazy gsc wjxof jyn
  fwzgsf jyn hjda jyn jyhszktcf jyn zdjyeigjyf jyn odcjvljfg hcdcjwf jyn lditg
  ojgf jyn gsc wzd fajvc fjqtie ltdfg fsjwg gszi gjvc zig gsc szwq aty gscy
  fsjwg gszi hziyg gz gsdcc yz xzdc yz wcff gsdcc fsjwg oc gsc yixocd gszi fsjwg
  hziyg jyn gsc yixocd zl gsc hziygtye fsjwg oc gsdcc lzid fsjwg gszi yzg hziyg
  yctgsd hziyg gszi gmz cphcagtye gsfg gszi gscy adzhccn gz gsdcc ltkc tf dtesg
  zig zyhc gsc yixocd gsdcc octye gsc gstdn yixocd oc dcjhscn gscy wzoocfg gszi
  gsq szwq sjyn edcyjnc zl jygtzhs gzmjdnf gszi lzc msz octye yjiesgq ty xq ftesg
  fsjww fyill tg"
4
5 # Letter frequency
6 letters = [c for c in encoded_msg if c.isalpha()]
7 freq = Counter(letters)
8
9 print("Total letters:", len(letters))
10 print("\nLetter frequencies (most common first):")
11 for letters, count in freq.most_common():
12     print(f" {letters}: {count}")
13
14 # Word frequency
15 words = encoded_msg.split()
16 word_freq = Counter(words)
17
18 print("\nMost common words:")
19 for word, count in word_freq.most_common(15):
20     print(f" {word}: {count}")
```

Total letters: 635

Letter frequencies (most common first):

g: 79
c: 68
s: 59
y: 55
j: 48
z: 47
f: 36
t: 34
d: 32
n: 29
i: 27
w: 22
o: 17
e: 15
h: 14
q: 12
l: 11

```
x: 10
a: 9
m: 5
v: 3
k: 2
p: 1
```

Most common words:

```
jyn: 11
gszi: 9
gsc: 9
fsjwg: 6
gsdcc: 5
hziyg: 4
yixocd: 4
sjyn: 3
edcyjnc: 3
wzdn: 3
gsq: 3
gz: 3
gscy: 3
oc: 3
fjqtye: 2
```

Most common words:

jyn: 11 <- likely to be "and"

gszi: 9 <- likely to be "thou"

gsc: 9 <- likely to be "the"

fsjwg: 6 <- likely to be "shalt"

▼ Initial Observations

The most frequent 3-letter words are 'gsc' and 'jyn'. In English, the most common 3-letter words are **the** and **and**.

- 'gsc' = the -> g:t, s:h, c:e
- 'jyn' = and -> j:a, y:n, n:d

other frequent words:

- 'gyzi' (4 letters, starts with gs) -> likely "thou"
- 'gstf' (4 letters, starts with gs) -> likely "this"
- 'syjn' (4 letters) with s:h, j:a, y:n, n:d = "hand"

```
1 # Initial mapping
2 mapping = {
3     'g':'t','s':'h','c':'e', # gsc = the
4     'j':'a','y':'n','n':'d', # jyn = and
5     'z':'o','i':'u',      # gszi = thou
6     't':'i','f':'s', # gstf = this
7     'd':'r', # gsdcc = three
8 }
9
10 def decode(msg, mapping):
11     result = []
12     for ch in msg:
13         if ch in mapping:
14             result.append(mapping[ch])
15         elif ch == ' ':
16             result.append(' ')
17         else:
18             result.append('?') # unknown
19     return ''.join(result)
20
21 print(decode(encoded_msg, mapping))
```

and st atti?a raised his hand ?renade u? on hi?h sa?in? o ?ord ??ess this th? hand ?renade that ?ith it thou ?a?est ??o? thine e

▼ Additional mappings discovered

From partially decoded text I can now identify more words:

-'edcyjnc' = grenade -> e:g, d:r(already), c:e, y:n, j:a, n:d
 -'wzdn' = lord -> w:l
 -'aczawc' = people -> a:p, w:l
 -'lcjfg' = feast -> l:f
 -'xcdhq' = mercy -> x:m, h:c, q:y
 -'yixocd' = number -> x:m, o:b, d:r
 -'zig' = out -> z:o, i:u
 -'gstdn' = third -> g:t, s:h, t:i, d:r, n:d
 -'szwq' = holy -> s:h, z:o, w:l, q:y
 -'hziyg' = count -> h:c, z:o, i:u, y:n, g:t
 -'wjxof' = lambs -> w:l, j:a, x:m, o:b, f:s
 -'fwzgsf' = sloths -> f:s, w:l, z:o, g:t, s:h, f:s
 -'jyhszktcf' = anchovies -> k:v
 -'odcjljfg' = breakfast -> v:k, l:f
 -'hdcjwfi' = cereals -> h:c, c:e, d:r, c:e, j:a, w:l, f:s
 -'wzoocfg' = lobbest -> w:l, z:o, o:b, c:e, f:s, g:t
 -'yjiesgq' = naughty -> y:n, j:a, i:u, e:g, s:h, g:t, q:y

```

1 # Complete mapping (encoded -> plaintext)
2 mapping = {
3     'a': 'p',
4     'c': 'e',
5     'd': 'n',
6     'e': 'g',
7     'f': 's',
8     'g': 't',
9     'h': 'c',
10    'i': 'u',
11    'j': 'a',
12    'k': 'v',
13    'l': 'f',
14    'm': 'w',
15    'n': 'd',
16    'o': 'b',
17    'q': 'y',
18    's': 'h',
19    't': 'i',
20    'v': 'k',
21    'w': 'l',
22    'x': 'm',
23    'y': 'n',
24    'z': 'o',
25 }
26
27 def decode(msg, mapping):
28     return ''.join(mapping.get(ch, ch) if ch != ' ' else ' ' for ch in msg)
29
30 decoded_message = decode(encoded_msg, mapping)
31 print(decoded_message)

```

and st attila raised his hand grenade up on high saying o lord bless this thy hand grenade that with it thou mayest blow thine e

✓ Final Decoded Message

and st attila raised his hand grenade up on high saying o lord bless this thy hand grenade that with it thou mayest blow thine enemies to tiny bits in thy mercy and the lord did grin and people did feast upon the lambs and sloths and carp and anchovies and orangutans and breakfast cereals and fruit bats and the lord spake saying first shalt thou take out the holy pin then shalt thou count to three no more no less three shalt be the number thou shalt count and the number of the counting shalt be three four shalt thou not count neither count

thou two ecepting that thou then proceed to three five is right out once the number three being the third number be reached then
lobbest thou thy holy hand grenade of antioch towards thou foe who being naughty in my sight shall snuff it

```
1 with open('decoded.txt', 'w') as f:  
2     f.write(decoded_message)  
3  
4 print("Saved to decoded.txt")  
5 print(decoded_message)
```

Saved to decoded.txt
and st attila raised his hand grenade up on high saying o lord bless this thy hand grenade that with it thou mayest blow thine e

```
1 # Print the completely decoded message  
2 print(decoded_message)
```

and st attila raised his hand grenade up on high saying o lord bless this thy hand grenade that with it thou mayest blow thine e